



Red Hat Limited | Building 6700 | Cork Airport Business Park | Kinsale Road | Cork | T12 XR60 | Ireland | redhat.com
Company ID Number: 304873

Date: 01/10/2026

To the Editors, Journal of Object Technology

Re: Submission of "The Assertion Ladder: Graduated Assertion for Embedding a Technical Domain in Repositories Shared with Coding Agents" as a regular research article

Dear Editors,

Please consider the attached manuscript for publication in the Journal of Object Technology. I write as corresponding author on behalf of my co-authors Paul Power and Ray Carroll at Red Hat and Colm Dunphy, Darrin Taylor and Eamonn de Leastar at South East Technological University (SETU).

Why now. Agentic AI has changed who writes software. Coding agents read a repository, plan a change, edit many files and run the tests before a person sees the result. They are capable, and they have no memory: every session starts again from nothing, so the repository is the only thing an agent and a team reliably share. Changes also arrive faster than people can review them closely. Unless the repository itself teaches and enforces a domain's rules, those rules spread and erode. The question the paper answers is the same for a student, a new hire and an agent: how does a repository make its domain rules impossible to miss and hard to break, without burying everyone in prose?

Where the thinking comes from. The Lean reading at the heart of the paper is one I set out at book length in *Build High Value Software Systems* (Manning, Early Access; publication expected November 2026). The book argues that Lean's answer to work done quickly by many hands applies to software, and is needed more now that so much code is produced so fast. The paper takes one piece of that argument, the repository, and works it through: value, standard work, the value stream and root cause analysis, with the Assertion Ladder as standard work, with its tests built in, for a contributor who cannot remember it. The book is a practitioner text; the manuscript shares its vocabulary and nothing else.

The collaboration. This is joint work between SETU and Red Hat in Waterford, Ireland, and we meet the problem from both sides. Two of my co-authors lead SETU's online Higher Diploma in Science in Computer Science, where students now learn to program alongside agents and lecturers need codebases that teach a domain to both. A third leads SETU's Master of Business Studies in Lean Enterprise Excellence, which teaches the Lean practice the paper draws on. Three of us work at Red Hat, where the same agents contribute to engineering work and hiring is moving towards graduates who can work safely in repositories that agents also change. An agent with no limits builds past what a student can yet read and past what a team can hold in mind. We meet that as a culture and as an industry, and the paper treats it as one problem.

Contribution. Current practice combines architecture conformance checks, executable specifications and instruction files for agents in an ad hoc way. The paper identifies five gaps that call for standardisation: no shared vocabulary for how strongly a rule is asserted; agent instructions with no link to enforcement; no guidance on which rules to check rather than document; requirements and scenarios not tied together; and no benchmark of whether a repository's own checks catch the mistakes that matter. It proposes the **Assertion Ladder** as that standard: six rungs, from a

rule that is merely implied to one that cannot be written wrongly, realised by nine smaller patterns, with an agentic loop of trust, delegation and enforced conformity that keeps a team climbing.

Tutors is at the centre of the work. Tutors is SETU's open-source learning platform, a production TypeScript codebase that coding agents were already changing. The paper shows the ladder retrofitted to it over one summer: architecture rules, bound scenarios, requirements in EARS form, shrink-only ratchets and mutation testing, all added after the fact, and its release checks extracted into a reusable release harness that the change under review cannot weaken. The harness is young and the paper says so: each new check enters as informing and earns the right to block, which is the ladder applied to the harness's own checks. We show intent rather than perfection. A small greenfield service, presented in TypeScript with Java and Python implementations shown to be equivalent, shows the same ladder designed in from the start.

Evidence. Violation seeding, used as a benchmark of the repository rather than of the agent, compares two iterations of the greenfield service: the first, enforcing structure alone, caught 30 of 36 seeded mistakes; the second, with the behavioural and requirement rungs, catches all 42. A pilot with eighteen coding agents suggests that the declared and enforced rungs reduce convention drift. The repositories were built by the authors working with coding agents, in the collaborative mode the paper studies; the manuscript says so in its threats to validity, and the repository history is left intact as part of the record. All code, data and scripts are public.

The work sits within the Journal's scope in software architecture, requirements engineering, validation and verification, and software process. The manuscript has not been published and is not under consideration elsewhere. All six authors have approved the submission, and we are happy for the article to appear under a Creative Commons Attribution licence.

Thank you for your consideration.

Yours sincerely,
Leigh Griffin
Senior Engineering Manager
Red Hat Limited
lgriffin@redhat.com
